COMPRESSION

BY

DOUGLAS QUIGG

THE FLOOR THE CODER ANSWERS TO
EVERY CLAIM, WHAT IS BEHIND IT, AND WHAT IT WANTS

2026

© 2026 Douglas Quigg
AGPL-3.0-or-later OR LicenseRef-Commercial OR LicenseRef-Educational

CONTENTS

1 THE WORKBOOK IS THE LEDGER 1
1.1 How an entry is retained 1
1.2 Entries 1

2 THE COMPRESSION TABLE 3
2.1 The lower bound 3
2.2 The table's algebra 5
2.2.1 C0. Exact counts (F1, F2, F3, F5) 5
2.2.2 C1. What one frame costs, given everything else (F4) 5
2.2.3 C2. The two-part code (F3′, F4) 5
2.2.4 C3. The frame difference removes the fixed pattern (the photon transfer curve) 5
2.2.5 C4. The second difference removes linear drift (the photon transfer curve) 5
2.2.6 C5. The neighbor removes smooth structure (F4 by neighbor differencing) 6
2.2.7 C6. Correlation separates noise from structure (F4 by neighbor differencing) 6
2.2.8 C7. The photon transfer curve (F4) 6
2.2.9 C8. The robust spread (the photon transfer curve) 6
2.2.10 C9. The gap in units (every row's gap) 6
2.2.11 C10. No combination of frames goes below their draws (F4) . . 6
2.2.12 C11. A reversible computation conserves bits (F4, the compressed file) 6
2.2.13 C12. Linear complexity (the low planes) 7
2.2.14 C13. The coder against the count (F5) 7
2.2.15 C14. Charge before the gain sits on the photon transfer line (F4; rows 2 to 6) 7
2.2.16 C15. The cumulant slopes of a gained count (F4, F5; rows 1, 12 and 15) 7
2.2.17 C16. Thinning retains a Poisson count Poisson (rows 13, 14 and 16) 8
2.2.18 C17. The structure function over lags (rows 7 and 10) 8
2.2.19 C18. The row and column sums (rows 8, 9 and 10) 8
2.2.20 C19. The single-electron tail (row 11) 9
2.2.21 C20. Crosstalk in the neighbor correlation (rows 16 and 17) . . 9
2.2.22 C21. A pointwise step is its table (F3′; the table step) 9
2.3 The noise floor on 44b6_0113de3b (23 September) 10

2.4 The noise floor on the 25 samples (23 September) . . . 12
2.5 The plan: noise departure by functionals (23 September) . . . 14
2.6 The table . . . 16
2.7 What has been tried on the coefficients . . . 17
2.8 Built in the engine, not yet in the compression path . . . 18
2.9 How to close the gap, in order . . . 18

3 THE BUILD PLAN: COMPRESSION . . . 21
3.1 Found, 24 September . . . 21

4 LEDGER . . . 23
4.1 2026-09-21 . . . 23
4.2 The .kcr codec . . . 23
4.3 The residual's code length: radix around a line, rotation, spiral . . . 23
4.4 The e − 1 ratio . . . 24
4.5 2026-09-23 . . . 24
4.6 A synthetic test of the lower bound . . . 24
4.7 The noise floor on 44b6_0113de3b . . . 25
4.8 The noise floor on the 25 samples . . . 26

BIBLIOGRAPHY . . . 29

1

THE WORKBOOK IS THE LEDGER

Purpose: One place where every idea the compression lower bound rests on is written down beside what is behind it. A reader then knows at a glance which claims are proved, which are measured, which are built but unmeasured, and which are still only theory.

Scope: `workbooks/compression/`. The coder and the hash tree are machine parts, in the engine workbook, `workbooks/engine/` (M9 and M13 of its engine_table.md).

1.1 HOW AN ENTRY IS RETAINED

Every claim carries one status, and the status says what backs it:

status	what it means
proved	an exact check ran over the whole set and found no exception; the count is given
measured	a number was taken on named samples; the number is given, and whether it survived
built	the code exists and runs; no measurement yet says whether it helps
theory	stated, not built
refuted	measured, and the measurement went against it; retained, with the number, and not tried again blind
not so	fails on its own arithmetic or in any machine, before anything is measured; retained, with the reason

A claim changes status only when a run changes it, and the run is named. Nothing is deleted from the ledger. An idea that failed stays, with the number that failed it.

1.2 ENTRIES

file	what it holds
compression_table.md	the compressed file's size dataset by dataset against the lower bound the data allows: the hierarchy of exact bounds from raw to the noise floor, every coder variant tried, and the order to close the gap
build_plan.md	the compression item of the build plan

file	what it holds
ledger.md	every measurement of the .iapx codec, its variants and the lower bound it answers to, in the order it was taken, with its samples and its result

2

THE COMPRESSION TABLE

Purpose: Track, dataset by dataset, the size of the compressed file against the lower bound the data allows. The gap between the two is always a number, and every change to the coder is judged against that gap.

Scope: The compressed file, `.kcr`, and what it holds: the lifting transform's coefficients, the entropy-coded stream, the deflated side information, and the hash tree that seals it. The coder is M9 of engine_table.md, and the hash tree is M13. Measurements come from ledger.md, plus the RSNA project's own runs where they are named. Statuses follow README.md.

2.1 THE LOWER BOUND

The true lower bound is the Kolmogorov complexity [1] K(x) of a dataset: the length of the shortest program that prints it. K is not computable, and it cannot be measured directly. What can be measured is a hierarchy of bounds, each one exact for a named class of coder, and each lower than the one before as the coder is allowed to see more. The compressed file's size is judged against the bound for the class its coder belongs to, and the next bound down shows what a better coder could still take.

name	the bound	identity	exact for	status
F0. Raw	the source's own bytes: 2 bytes a voxel at 16 bits a sample	none	nothing; this is the reference every percentage is taken of	measured on every set
F1. Values, zeroth-order empirical entropy ($n \cdot H_0$ in exact combinatorial form)	⌈log2(n! / Π_v c_v!)⌉ bits over a sample's n voxels, where c_v counts the voxels holding value v, plus the bits needed to state the counts	Co	any coder that treats the voxels as a multiset of values and sees no position	theory
F2. Coefficients, zeroth-order, per decomposition level	Σ_f ⌈log2(n_f! / Π_v c_{f,v}!)⌉ over the lifting transform's decomposition levels f, plus each level's counts	Co	a coder that sees the lifting transform's coefficients level by level, each level memoryless. The Rice coder belongs to this class, and F2 is the bound it is judged against	theory

name	the bound	identity	exact for	status
F3. Coefficients in context, k-th order empirical entropy	the same count, taken per context: a coefficient's value given its neighbors already coded (the parent level, the same place one frame back, the row before)	C0	context coders: a coefficient coded given its neighbors	theory
F3′. A two-part code	the size of an exact representation of the set (its tables and sums, kolmogorov_arnold.md) plus F2 of what that representation leaves	C2, C21	a coder that stores a model and then the residual the model does not predict; its length is an upper bound on K(x)	theory; its tables are the lookup-table step (C21), proved in the engine (M10, A13; the section on built parts below), and the code itself is not measured
F4. The noise floor	Σ over voxels of the entropy of that voxel's noise given everything else known: the dataset's deterministic terms, the spatial correlation, and the signal level the voxel sits at. For noise of spread σ that is about $\log_2(\sigma\cdot\sqrt{(2\pi e)})$ bits a voxel, however many low bit planes look like coin flips	C1, read through C3 to C8 and C14 to C20; bounded by C10 and C11	every coder: what is left of one frame after all structure is taken, the dataset's model paid once and so costing nothing per frame in the limit	measured on the 25 samples: 6.229 bits a voxel, 38.9% of raw (31.8% to 43.6% by sample), from the photon transfer curve with structure removed by neighbor differencing (sections below)
F5. The integrated noise functionals	Σ over voxels of the code length of each voxel's residual under the noise the category functionals describe there (next section): the source's own entropy once the functionals are the real categories	C0, C13	a coder driven by the functionals' exact counts: exact arithmetic coding comes within 2 bits of this length over a whole stream, and enumerative coding in exact integers hits it exactly	theory: the plan below

Each bound is an integer count of bits, and each is a bit length of an exact integer (a multinomial coefficient). Stating it needs no logarithm or float. The cost is size: at 419,430,400 voxels a sample, the multinomial runs to billions of bits. Computing F1 to F3 exactly at that size is open: the tool is not built, and neither is its method.

What F4 says. On all 25 44b6 samples, bits 0 to 3 flip 499 to 500 times per thousand transitions in every window of 19 samples, and 350 to 499 in the other 6. Counting those bit planes as the noise floor, near 25% to 31% of raw, undercounts: bits 4 and 5 are coin flips too (below), and counting uniform bit planes is not the noise floor in any case. The noise floor is the entropy of each voxel's noise at its own signal level, measured below on 44b6_0113de3b.

2.2 THE TABLE'S ALGEBRA

Every bound and every accepted reading holds to one of these identities, as each engine member holds to its A-number in engine_table.md. A reading that cites none is not accepted. Write a voxel's value in frame t as I_t = P + S_t + n_t: P its fixed-pattern noise (DSNU and PRNU), S_t the structure (the cells and whatever the tracks carry), and n_t the temporal noise, independent between frames and between voxels, mean 0, variance $\sigma^2(L)$ at signal level L. In C14 to C20, a row is a row of the noise vector integration table.

2.2.1 *Co. Exact counts (F1, F2, F3, F5)*

n symbols with counts c_v have exactly n! / Π_v c_v! arrangements. The enumerative code (the stream's index among them) spends $\lceil \log_2(n! / \Pi_v c_v!) \rceil$ bits, and no code that sees only the counts spends less. It is the bit length of an exact integer, with no logarithm formed. F1 takes it over the values, F2 per decomposition level, F3 per context, F5 per signal level under the functionals.

2.2.2 *C1. What one frame costs, given everything else (F4)*

Integer values whose noise has spread $\sigma \gg 1$ carry $H \approx \frac{1}{2} \log_2(2\pi e\, \sigma^2)$ bits (a Gaussian's entropy at unit step). For one frame of V voxels:

$F_4 = \Sigma_v \frac{1}{2} \log_2(2\pi e\, \sigma^2(L_v))$ bits,

and per voxel the mean of that sum. The Gaussian form is the working estimate; Co taken per level is its exact form (F5).

2.2.3 *C2. The two-part code (F3′, F4)*

For N frames, a code spends $M + N \cdot V \cdot F_4$ bits, with M the model: P, the gain, the read noise, the tracks. Per frame that is $M/N + V \cdot F_4$, which goes to $V \cdot F_4$ as N grows. The model is the short part; the frame's own random draw is the rest.

2.2.4 *C3. The frame difference removes the fixed pattern (the photon transfer curve)*

$d = I_{t+1} - I_t = \Delta S + (n_{t+1} - n_t)$. P cancels exactly, and $\mathrm{var}(d) = \mathrm{var}(\Delta S) + 2\sigma^2$.

2.2.5 *C4. The second difference removes linear drift (the photon transfer curve)*

$e = 2I_t - I_{t-1} - I_{t+1} = (2S_t - S_{t-1} - S_{t+1}) + (2n_t - n_{t-1} - n_{t+1})$, with noise variance $4\sigma^2 + \sigma^2 + \sigma^2 = 6\sigma^2$. Where S moves linearly in t, e holds noise only, and $\mathrm{var}(e)/6 = \sigma^2$.

2.2.6 *C5. The neighbor removes smooth structure (F4 by neighbor differencing)*

For x the next voxel along a row, $d - d_x = (\Delta S - \Delta S_x)$ + four independent noise terms, and $\mathrm{var}(d - d_x) = \mathrm{var}(\Delta S - \Delta S_x) + 4\sigma^2 \geq 4\sigma^2$. So $\sigma^2 \leq \mathrm{var}(d - d_x) / 4$, with equality where the structure changes the same at both voxels: an upper bound that tightens as the structure smooths.

2.2.7 *C6. Correlation separates noise from structure (F4 by neighbor differencing)*

White noise has no covariance with its neighbor, which gives $\mathrm{corr}(d, d_x) = \mathrm{cov}(\Delta S, \Delta S_x) / (\mathrm{var}(\Delta S) + 2\sigma^2)$. It is 0 where there is no structure and goes to 1 where structure dominates.

2.2.8 *C7. The photon transfer curve (F4)*

Shot noise counts photons, and its variance is proportional to the signal: $\sigma^2(L) = g \cdot (L - O) + R^2$, with g the overall system gain, O the dark offset and R the read noise. A line fitted as $\sigma^2 = g \cdot L + c$ has $c = R^2 - g \cdot O$ and meets zero at $L_0 = -c/g$. If $O = 0$, then $c = R^2$.

2.2.9 *C8. The robust spread (the photon transfer curve)*

For Gaussian noise, the median absolute deviation × 1.4826 = σ. A cell that crosses a voxel in a few of its frames moves the mean and the plain variance, but not the median.

2.2.10 *C9. The gap in units (every row's gap)*

A stream of B bits a voxel sits $(B - F_4) / 16$ of raw above the noise floor, and $V \cdot (B - F_4) / 8$ bytes above it per frame.

2.2.11 *C10. No combination of frames goes below their draws (F4)*

N frames of independent k-bit draws have 2^(N·k) possible values. A k-bit summary (an AND, an OR, an XOR, a sum) names only 2^k of them. A lossless code retains the other $(N - 1) \cdot k$ bits somewhere. What every frame shares is read once: ∧_t I_t gives the bits set in every frame, and ∧_t ¬I_t the bits clear in every frame. Those are the bits constant across frames, the engine's anchor bits.

2.2.12 *C11. A reversible computation conserves bits (F4, the compressed file)*

Every lifting step is an integer bijection (A8). Running the scheduler's log backward or forward maps each state to exactly one state and retains its bit count. The log holds the operations (few bits) and the random draw ($V \cdot F_4$ a frame). It shrinks to the operations alone only when the draw came from a program, as in the synthetic test, where the draws recovered to a 64-bit xorshift seed.

2.2.13 *C12. Linear complexity (the low planes)*

A random sequence of n bits has linear complexity L near n/2, and L ≤ n/2 − k with probability about 2^(−2k). The "low" mark at k = 8 is about 2^{-16}. A linear generator with s bits of state has L ≤ s for every n.

2.2.14 *C13. The coder against the count (F5)*

Exact arithmetic coding spends at max $\lceil \Sigma - \log_2 p \rceil + 1$ bits over a whole stream, within 2 bits of the model's length. Enumerative coding (C0) spends exactly the count. A fixed-width range coder rounds each interval and loses a sliver per symbol.

2.2.15 *C14. Charge before the gain sits on the photon transfer line (F4; rows 2 to 6)*

Let a voxel collect, each frame, μ photoelectrons, D·Δt dark electrons and ε spurious ones, each a Poisson count and all before the gain. Let the offset O be added after the gain, with the signal-independent terms after the draw (read R^2, thermal σ^2_J, reset σ^2_kTC) independent and of mean 0. With one gain g for every electron,

$\bar{L} = O + g\cdot(\mu + D\cdot\Delta t + \varepsilon)$, $\sigma^2 = g^2\cdot(\mu + D\cdot\Delta t + \varepsilon) + R^2 + \sigma^2_J + \sigma^2_kTC$,

giving $\sigma^2 = g\cdot(\bar{L} - O) + R^2 + \sigma^2_J + \sigma^2_kTC$ exactly. Charge that enters as whole electrons before the gain moves a voxel along the transfer line and never off it. At one exposure, the dark level, the dark shot and spurious charge are read as light, and the line's intercept (C7), $c = R^2 + \sigma^2_J + \sigma^2_kTC - g\cdot O$, holds none of them. Dark current is read only where the light is known to be zero. A bias frame (no light, no exposure) gives O as its mean and the signal-independent sum as its variance. A dark frame at Δt adds g·D·Δt to the mean and g^2·D·Δt to the variance, and their ratio is g. No frame at one temperature splits R^2, σ^2_J and σ^2_kTC from each other.

2.2.16 *C15. The cumulant slopes of a gained count (F4, F5; rows 1, 12 and 15)*

Let each electron give an independent output X in digital numbers (DN), of one law for every electron, and let the count of electrons be Poisson with mean $\bar{n}$. The output is a compound Poisson sum, whose cumulants are $\kappa_r = \bar{n}\cdot E[X^r]$ for every $r \geq 1$. Level-free terms added after the draw (C14) change no slope, and symmetric ones add 0 to every odd cumulant. Against the level, each cumulant is a line of slope

$s_r = E[X^r] / E[X]$,

giving $s_1 = 1$ and $s_2 = g\cdot F^2$, with g = E[X] the gain and $F^2 = E[X^2] / E[X]^2$ the excess noise factor (row 12). The transfer curve's slope (C7) is $g\cdot F^2$, larger than g by the excess noise factor. One output for every electron (X = g) gives $s_r = g^{(r-1)}$: $F^2 = 1$, and the values g, g^2, g^3 the cumulant slopes are graded against (row 1). An electron-multiplying register's output for one electron is near exponential, with $E[X^r] = r!\cdot g^r$. Then $F^2 = 2$, $s_2 = 2g$, $s_3 = 6g^2$ and $s_3 = 3g\cdot s_2$.

The slopes are log-convex. For X ≥ 0, Cauchy–Schwarz gives $E[X^2]^2 \leq E[X]\cdot E[X^3]$ and $E[X^3]^2 \leq E[X^2]\cdot E[X^4]$, and therefore

$s_3 \geq s_2^2$, $s_2\cdot s_4 \geq s_3^2$,

each an equality only where every electron gives the same X. Both are cross-multiplied comparisons of fitted slopes, with no root or quotient formed. A set of slopes that breaks either is not one gained Poisson count plus symmetric signal-independent noise. The slope s_2 alone never splits g from F^2; s_2 and s_3 together split them only once the single electron's law is named.

2.2.17 *C16. Thinning retains a Poisson count Poisson (rows 13, 14 and 16)*

If N is Poisson with mean λ and each of its events is retained independently with probability q, the retained count is Poisson with mean q·λ, and the retained and lost counts are independent: the generating function exp(λ·(z − 1)), taken at 1 − q + q·z, is exp(q·λ·(z − 1)). Quantum efficiency (row 14) and partition (row 13) thin the light. On Poisson light they only rescale its mean, and no reading of the frames names Q_e or a partition apart from the light's own rate. For any count with Fano factor f = var(N) / E[N], the thinned count's is

f′ = 1 + q·(f − 1),

A thinning moves a count toward Poisson and never away, and partition is read only on a light whose f ≠ 1. The same identity splits a Poisson count among places. Photons scattered into the neighboring pixels before the draw (optical crosstalk, row 16) leave each pixel's count an independent Poisson count at its own blurred rate.

2.2.18 *C17. The structure function over lags (rows 7 and 10)*

For noise stationary in t with autocovariance γ(k) at lag k, the structure function D(k) = E[(n_{t+k} − n_t)²] is 2·(γ(0) − γ(k)). White noise holds D(k) = D(1) = 2σ² at every lag k ≥ 1, and a random walk of step variance q holds D(k) = k·q. An octave lasting 2° frames, redrawn at every multiple of 2° with variance v, adds 2v·min(k, 2°) / 2°, averaged over the frame's phase: it grows linearly to lag 2° and holds there.

Taken less the x neighbor's difference, D_x(k) = E[(d_k − d_{k,x})²] with d_k = I_{t+k} − I_t. A term both voxels share additively cancels exactly, and each voxel's own terms add: for independent noise of one law, D_x(k) = 2·D(k). A scale a_t shared by the light (row 10) does not cancel. It adds (L − L_x)²·D_a(k), with D_a the scale's own structure function, which is zero only where the two levels are equal.

2.2.19 *C18. The row and column sums (rows 8, 9 and 10)*

Take one frame pair's differences d over a set of voxels (the static ones). Let each be d = r + c + p + e, with r shared by the voxel's row, c by its column, p by its whole plane and e its own, all independent and of mean 0, with variances σ²_r, σ²_c, σ²_p and s. For two distinct voxels, E[d·d′] is σ²_r + σ²_p on the same row, σ²_c + σ²_p on the same column, and σ²_p otherwise. Write T = Σ d² over the M voxels. A squared sum less its own squares is the sum of its ordered pairs' products. Take A_row = Σ over rows of (Σ d)², less T; A_column likewise; A_plane = Σ over planes of (Σ d)², less T; and K_row, K_column and K_plane the ordered pairs each holds. Then

E[A_row] = K_row·(σ²_r + σ²_p), E[A_column] = K_column·(σ²_c + σ²_p), E[A_plane − A_row − A_column] = (K_plane − K_row − K_column)·σ²_p, E[T] = M·(σ²_r + σ²_c + σ²_p + s),

for any count of voxels on any row or column, balanced or not. Each is a sum of integer products, and each variance follows by cross-multiplying over the common denominator M·K_row·K_column·(K_plane − K_row − K_column), as `noise_static_readings` does. The same three lines are E16's patterns ROWS, COLUMNS and PLANES in engine_table.md.

2.2.20 *C19. The single-electron tail (row 11)*

Charge that enters as whole electrons before the gain joins the light's count (C14) and sits on every cumulant's line through the offset. With the signal-independent noise symmetric, every odd cumulant is then zero at the offset:

$\kappa_3 = s_3\cdot(\bar{L} - O)$, $\kappa_2 = s_2\cdot(\bar{L} - O) + R^2$,

with R^2 the whole signal-independent sum of C14. The two lines' intercepts c_2 and c_3 give $O\cdot s_3 = -c_3$ and $R^2\cdot s_3 = c_2\cdot s_3 - c_3\cdot s_2$, cross-multiplied: the offset and the signal-independent sum apart, from frames at one exposure. A charge that does not enter as a whole electron before the gain (one made partway through the gain, which sees less of it) has its own output law X' and rate ε', both signal-independent. It adds $\varepsilon'\cdot E[X']$ to the level and $\varepsilon'\cdot E[X'^r]$ to every cumulant. The offset κ_3 reads is $O + \varepsilon'\cdot(E[X'] - E[X'^3] / s_3)$.

Spikes against dips read the same odd part. Where the joint law of the three frames around one is unchanged by the reflection $x \mapsto 2c - x$, the reflection maps each spike at threshold h to a dip at h, and the expected counts are equal. The shot's own skew ($s_3 > 0$), a sparse tail and a cell passing in one frame can each break that, and the two counts alone don't say which: row 11 is not separable here.

2.2.21 *C20. Crosstalk in the neighbor correlation (rows 16 and 17)*

Electrical crosstalk shares a fraction α of each voxel's draw with its two neighbors along one axis after the draw: $d'(v) = d(v) + \alpha\cdot(d(v - \delta) + d(v + \delta))$. For d independent between voxels, with variance V,

$\mathrm{var}(d') = (1 + 2\alpha^2)\cdot V$, $\mathrm{cov}(d'(v), d'(v + \delta)) = 2\alpha\cdot V$, $\mathrm{cov}(d'(v), d'(v + 2\delta)) = \alpha^2\cdot V$,

and 0 past two steps. The correlation is $2\alpha / (1 + 2\alpha^2)$ at one step and $\alpha^2 / (1 + 2\alpha^2)$ at two, the same at every level, since it is a fixed fraction of whatever was drawn. With the crosstalk along x and $V = 2\sigma^2$, the neighbor route (C5) reads

$E[(d' - d'_x)^2] / 4 = \sigma^2\cdot(1 - 2\alpha + 2\alpha^2)$,

where the raw route $E[d'^2] / 2$ reads $\sigma^2\cdot(1 + 2\alpha^2)$. A positive α lowers the neighbor route's slope and intercept by the same factor. Optical crosstalk scatters light before the draw, and by C16 the pixels' counts stay independent. It adds no correlation at any reach and shows only in the signal's edge spread (row 16). A correlation of the noise between neighbors (row 17) is sharing after the draw, and one that holds past the second step is not crosstalk of this form.

2.2.22 *C21. A pointwise step is its table (F3′; the table step)*

Let f map the lane's alphabet $A = \{0, \ldots, 2^b - 1\}$ to w-bit values. Its table T_f holds f(a) at row a for every a in A. One pass of f over A fills it: 2^b evaluations, however many lanes it is read

on afterward. For every lane x, T_f[x] = f(x) exactly, because the table is f's graph on A and x lies in A. One table read through another is their composition, T_g[T_f[x]] = g(f(x)), wherever f's values lie in g's alphabet. A chain of steps therefore folds into one table in program order. The fold regroups and does not reorder, since g∘f and f∘g differ in general. The table costs 2^b·w bits to state, once for the set. In the two-part code (C2) it sits in the model M, whose share per frame goes to 0 as the frames grow. Inside the reversible stream (C11), a table must be a bijection on its index bits. Any other f is made reversible by the embedding (x, y) ↦ (x, y ⊕ f(x)), which is its own inverse for every f.

2.3 THE NOISE FLOOR ON 44B6_0113DE3B (23 SEPTEMBER)

All from the frames themselves, read through `cell_tracking/maint/zarr_frames.py`. The scripts are scratch (`linear_complexity.py`, `conditional_floor.py`, `photon_transfer.py`, `coherence.py`), not in the tree.

No linear generator in the low planes (C12). Berlekamp–Massey gives the exact shortest linear feedback shift register over GF(2) that produces a bit sequence: its linear complexity L. Random bits give L near n/2. Any linear generator (xorshift, an LFSR, any linear congruential low bit) gives L at its state size, whatever n is. "Low" below means $L \le n/2 - 8$, which random bits reach with probability about 2^{-16}.

sequences	n	bits 0 to 5	bit 6	bits 7 to 10	bit 11
4,109 voxels in time (every 1021st)	100	mean L 50.22 to 50.24; 0 low	48.72; 201 low	43.48 to 19.43; 646 to 2,925 low	1.79; 4,024 low (almost constant)
1,024 rows of frame 50 (every 4th z and y)	256	128.14 to 128.26; 0 low	128.26; 0 low	127.83 to 78.82; 19 to 664 low	6.60; 996 low
4 raster runs of frame 50	8,192	4,096 to 4,097; 0 low	4,095.75; 0 low	4,096 to 4,543	0
controls: os.urandom	100 / 256	50.19 / 128.25; 0 low			
controls: xorshift64	256	exactly 64 on all 64 seeds: caught every time			

Bits 0 to 5 are indistinguishable from os.urandom in every direction the test reads. Structure begins at bit 6 and grows to bit 11. This rules out linear generators only; nonlinear ones are untested. The fixed-pattern noise does not show here: it is an offset added below the noise, which scrambles into the low bits. The per-voxel mean reads it, and bit linearity cannot.

The noise at each level (the photon transfer curve, first pass; C3, C1). The variance of each voxel is taken from successive frame differences ($\Sigma (I_t - I_{t-1})^2 / 2(n - 1)$), grouped by the voxel's mean over the 100 frames:

decile	mean level	variance	σ	$\log_2(\sigma\cdot\sqrt{(2\pi e)})$ bits	variance / mean
0	47.2	57.8	7.6	4.97	1.22
1	79.9	190.4	13.8	5.83	2.38
2	119.3	831.1	28.8	6.90	6.97
3	152.2	1,276.3	35.7	7.21	8.39
4	192.8	2,109.6	45.9	7.57	10.94
5	244.6	4,190.4	64.7	8.06	17.13
6	305.5	9,689.8	98.4	8.67	31.72
7	371.1	15,631.4	125.0	9.01	42.12
8	434.3	28,994.8	170.3	9.46	66.76
9	523.0	47,819.6	218.7	9.82	91.43

Shot noise has variance proportional to the level: variance / mean constant. Here it climbs from 1.22 to 91.43. So above the dimmest decile the frame-to-frame change is mostly not noise: it is the signal moving, cells crossing the voxel between frames. That part is deterministic given the tracks, and the dataset's model and neighbor differencing remove it.

reading	bits a voxel	of raw
each voxel's own σ as measured, motion left in (upper bound on the temporal-only model)	7.403	46.3%
the same, empirical counts of the residual from the 100-frame mean, per σ bin	7.687	48.0%
one frame alone, residual from the mean of 4 neighbors, order 0 (uses neighbors on both sides and is not a code length a coder can spend; a reference only)	6.287	39.3%
shot-limited noise floor, first estimate: variance = 1.22 × level at every decile, the gain read from the dimmest decile	about 6.00	about 37.5%
the stream as the `.kcr` wrote it	6.49	40.6%

The first estimate left two things open, the dark offset and whether the gain holds at every level. The next two passes settle both.

The photon transfer curve, motion removed in time (C3, C4, C7, C8). Each frame difference I_t − I_{t − 1} is binned by its own level (the pair's mean, in bins of 16), not by the voxel's 100-frame mean, and σ is taken from the median absolute deviation. A cell crossing a voxel in a few frames drops out of it. 41,528 voxels (every 101st) × 99 differences. The script is scratch (`photon_transfer.py`).

level	24	40	56	72	88	104	120	200	312	408	504	600	1000
robust variance, first difference / 2	27.5	53.9	70.3	89.0	109.9	133.0	185.7	396.8	989.1	2,858.6	23,107.6	62,254.9	173,220.9
robust variance, second difference / 6	29.7	44.3	61.9	93.8	118.7	146.5	161.6	399.0	952.9	2,060.7	9,261.7	17,092.5	32,971.6
variance / level	1.15	1.35	1.26	1.24	1.25	1.28	1.55	1.98	3.17	7.01	45.85	103.76	173.22

From level 24 to 104 the ratio holds at 1.15 to 1.35 and both estimators agree: shot noise. Above 120 the change is present in max frames. The median cannot remove it, and the ratio climbs to a plateau near 170,000. A single line fitted over every bin (138.4 × level − 23,437) is not a photon transfer curve; the steep bins dominate it, and its noise floor (4.97 bits) is discarded.

The same, motion removed by neighbor differencing (C5, C6, C7). Whether the bright excess is noise or structure is read from space: white noise does not correlate with its neighbor, moving structure does. The frame difference d is correlated with its x, y and z neighbors, per level, over 10 frame pairs of whole frames (41,943,040 voxel-frames); subtracting the neighbor cancels the smooth structure, and var(d − d_neighbour) / 4 is the white noise per frame that remains, an upper bound. The script is scratch (`coherence.py`).

level	24	40	56	72	104	136	168	200	248	296	344	408	504	1000
corr x of d	0.007	0.031	0.090	0.178	0.348	0.462	0.586	0.680	0.772	0.822	0.865	0.913	0.948	0.985
corr y of d	0.012	0.039	0.098	0.186	0.353	0.468	0.592	0.686	0.776	0.823	0.864	0.911	0.945	0.984
corr z of d	0.116	0.123	0.176	0.258	0.399	0.493	0.589	0.657	0.712	0.734	0.758	0.787	0.819	0.916
frame variance, structure in	35.6	49.5	73.6	105.7	189.3	296.9	477.2	742.6	1,327.4	2,122.1	3,547.5	8,392.6	29,530.8	212,744.2
white, less the x neighbor	37.6	49.2	67.1	86.9	123.4	159.4	197.1	237.0	303.5	382.4	487.7	747.8	1,583.7	3,197.9

level	24	40	56	72	104	136	168	200	248	296	344	408	504	1000
white / level	1.57	1.23	1.20	1.21	1.19	1.17	1.17	1.19	1.22	1.29	1.42	1.83	3.14	3.20

The bright excess is structure: the difference's correlation with its neighbor climbs from 0.007 at level 24 to 0.985 at 1000. With the neighbor taken off, what is left follows one line, 1.17 to 1.23 × level from 40 to 200, the same gain the time pass found in the dim range. Fitted there: **variance = 1.162 × level + 2.46**. The line meets zero near level −2: the data carries no dark offset, and the read noise is about 1.6 digital numbers (DN). Above about 300 the remainder climbs again, because one neighbor cannot cancel a sharp edge; that is structure a better predictor takes instead of noise. The z correlation of 0.12 at the dimmest levels is the only departure from white noise, and it is not explained.

reading, 44b6_0113de3b	identity	bits a voxel	of raw
the frame difference, structure in	C3, C1	6.913	43.2%
less the neighbor, best axis, plain	C5, C1	6.008	37.6%
less the x neighbor, robust	C5, C8, C1	5.979	37.4%
the photon transfer line, 1.162 × level + 2.46, at every voxel-frame's level (F4)	**C7 fitted through C5, then C1**	**5.902**	**36.9%**
the stream as the `.kcr` wrote it; its gap to F4	C9	6.49	40.6%

What it says. One frame of 44b6_0113de3b, given everything else, costs 5.90 bits a voxel: shot noise at gain 1.162 and read noise near 1.6, with every other change in the frame spatially coherent and so structure. The stream spends 6.49, 0.59 bits a voxel above it: 3.7 points of raw, about 310 KB a frame (4,194,304 voxels), 30.9 MB over the sample. The functionals can still take that much on this sample. The photon transfer line is taken as holding above level 300, where it cannot be read through the structure; a camera whose noise departs from shot there would move the noise floor up.

2.4 THE NOISE FLOOR ON THE 25 SAMPLES (23 SEPTEMBER)

The same coherence pass on each of the 25 (10 frame pairs of whole frames a sample, 874 s for the 24, 0 failed). The gain and intercept are fitted to var(d − d_x) / 4 from level 40 to 200 (C5, C7); the shot-only noise floor is C1 on that line at every voxel-frame's level. The neighbor reading is the robust x-neighbor bound (C5, C8, C1), and "structure in" is the frame difference alone (C3, C1). The script that fits and integrates is scratch (`shot_floor.py`).

sample	gain	intercept	white / level, 40 to 200	corr x of d at level 40	shot-only F4	less the neighbor	structure in
44b6_0113de3b	1.162	2.46	1.172 to 1.230	0.031	5.902 (36.9%)	5.979 (37.4%)	6.913 (43.2%)
44b6_0b24845f	1.035	2.49	1.045 to 1.123	0.057	6.973 (43.6%)	6.963 (43.5%)	8.158 (51.0%)
44b6_0c582fdc	0.973	2.42	0.981 to 1.104	0.024	6.368 (39.8%)	6.457 (40.4%)	7.365 (46.0%)
44b6_0db75fae	1.027	4.09	1.048 to 1.137	0.093	5.518 (34.5%)	5.269 (32.9%)	6.443 (40.3%)
44b6_12dfb391	1.013	−11.74	0.875 to 1.105	0.107	6.533 (40.8%)	6.591 (41.2%)	7.653 (47.8%)

sample	gain	intercept	white / level, 40 to 200	corr x of d at level 40	shot-only F4	less the neighbor	structure in
44b6_144b256d	0.980	5.67	1.008 to 1.042	0.031	6.764 (42.3%)	6.719 (42.0%)	7.522 (47.0%)
44b6_1574802b	0.998	4.12	1.018 to 1.103	0.018	5.375 (33.6%)	5.399 (33.7%)	5.946 (37.2%)
44b6_18ced818	0.833	13.34	0.903 to 1.071	0.054	6.822 (42.6%)	7.076 (44.2%)	8.558 (53.5%)
44b6_1d530831	0.917	8.57	0.962 to 1.193	0.019	6.508 (40.7%)	6.604 (41.3%)	7.707 (48.2%)
44b6_24264f12	1.011	3.43	1.030 to 1.165	0.004	6.056 (37.9%)	6.140 (38.4%)	7.008 (43.8%)
44b6_267148e4	1.030	1.74	1.037 to 1.071	0.761	5.688 (35.6%)	5.219 (32.6%)	6.870 (42.9%)
44b6_2a2eff9f	1.007	7.24	1.046 to 1.212	0.002	6.445 (40.3%)	6.448 (40.3%)	7.396 (46.2%)
44b6_2f31fc2f	0.901	5.09	0.926 to 1.035	0.014	6.227 (38.9%)	6.261 (39.1%)	7.100 (44.4%)
44b6_33b596bf	0.961	4.28	0.984 to 1.137	0.002	6.062 (37.9%)	6.142 (38.4%)	7.057 (44.1%)
44b6_341df25f	0.978	0.22	0.962 to 1.006	0.016	5.085 (31.8%)	5.086 (31.8%)	5.366 (33.5%)
44b6_3a861e03	1.118	0.85	1.101 to 1.245	0.064	6.289 (39.3%)	6.273 (39.2%)	7.268 (45.4%)
44b6_3bb3690f	0.904	7.13	0.941 to 1.160	−0.003	6.541 (40.9%)	6.579 (41.1%)	7.265 (45.4%)
44b6_40c45f5a	0.855	8.64	0.897 to 1.095	0.008	6.517 (40.7%)	6.633 (41.5%)	7.540 (47.1%)
44b6_415c0a3a	1.068	4.91	1.092 to 1.198	0.011	6.513 (40.7%)	6.521 (40.8%)	7.350 (45.9%)
44b6_53f95252	0.821	21.08	0.928 to 1.110	0.071	6.707 (41.9%)	6.903 (43.1%)	8.291 (51.8%)
44b6_551a5dba	0.951	1.82	0.957 to 1.035	−0.006	6.781 (42.4%)	6.904 (43.1%)	8.155 (51.0%)
44b6_5740d24b	1.056	7.02	1.087 to 1.212	0.150	6.145 (38.4%)	5.900 (36.9%)	6.702 (41.9%)
44b6_587a1e22	1.037	3.35	1.052 to 1.105	0.057	5.394 (33.7%)	5.419 (33.9%)	6.118 (38.2%)
44b6_5f15d135	0.892	10.62	0.942 to 1.089	0.061	6.708 (41.9%)	6.766 (42.3%)	7.711 (48.2%)
44b6_668e0cc7	1.130	2.35	1.142 to 1.185	0.354	5.806 (36.3%)	5.823 (36.4%)	6.920 (43.2%)
mean of the 25	**0.986**				**6.229 (38.9%)**	**6.243 (39.0%)**	**7.215 (45.1%)**

What it says.

- **One sensor noise model across the dataset.** The gain runs 0.821 to 1.162, mean 0.986: about one count a photon, with the offset near zero on max samples. The noise floor varies by sample (31.8% to 43.6%) because the samples sit at different signal levels, not because the noise law changes.
- **The dataset's noise floor is 38.9% of raw.** Each sample is the same size, and the dataset's F4 is the mean. The compressed files hold the 25 samples at 42.0% (6.721 bits a voxel), 0.492 bits a voxel above the noise floor: 3.1 points of raw, about 645 MB of the 8.81 GB (C9).
- **Constant regions go below the photon transfer line.** On 44b6_odb75fae, 267148e4 and 5740d24b the neighbor bound (32.9%, 32.6%, 36.9%) falls under the shot-only noise floor

(34.5%, 35.6%, 38.4%). All three are samples the engine's ledger found holding a constant region: voxels that never change cost nothing, and the photon transfer line does not know them. The fixed-pattern functional takes that part, and F4 on those samples is the lower of the two.

- **A spatially correlated category.** On max samples the dim differences are nearly white (corr x at level 40 below 0.11 on 21 of the 25). 44b6_267148e4 (0.761), 668eocc7 (0.354) and 5740d24b (0.150) are not: something moves whole neighborhoods together at the dimmest levels, frame to frame. It is a category the functionals must write (the spatially correlated row of the plan), and it is not yet identified.
- **Two fits to check.** 44b6_12dfb391 fits a negative intercept (-11.74), and 53f95252 an intercept of 21.08 with the lowest gain (0.821). Both lines bend inside 40 to 200. A straight line is not quite their law.

2.5 THE PLAN: NOISE DEPARTURE BY FUNCTIONALS (23 SEPTEMBER)

There is much more room to claim, by refining the spatial noise terms and the search. The algorithm can reach the theoretical lower bound universally if noise departure is treated carefully, with functionals. Once the real categories of noise are known, each gets its functional, and all of them are integrated.

The claim, stated exactly. K(x) is not computable: no algorithm reaches it on every input. What a coder can reach is the entropy of the source's true noise: Shannon's bound [2], the max any coder takes on average. A coder whose per-voxel counts come from functionals that describe the real noise reaches that bound, to within a constant, on every dataset those categories cover. "Universal" means exactly that: universal over the datasets whose noise the category functionals describe. When a dataset needs a category the functionals lack, the gap to F5 names it.

The categories to write functionals for. The four noise vectors (noise_sieve_tower.md §16), each a deterministic functional of the sample, read and never assumed. Each is a sum of exact integers, with no float:

category	how it moves	its functional, as exact sums over the sample	what the engine already measures
fixed-pattern noise (DSNU, the offset per pixel, and PRNU, the gain per pixel)	constant in time at a voxel	per voxel, the bits that never flip, and the per-voxel constant under the frames' mean	the anchor bits: set in every frame, different in every sample (**measured**)
shot noise (photon counting)	independent every frame; its spread grows with the signal	the photon transfer curve: $\Sigma (I_t - I_{t-1})^2$ against ΣI_t per level of the signal, both exact integers; a slope proportional to the level is shot	**measured** on 24 samples of the .kcr dataset: the slope $g \cdot F^2$, rows 1, 12 and 15 of the noise vector integration table, from the exact sums of `noise_detector` (`noise_flicker_set`); the fit is scratch (`transfer_residual.py`)

category	how it moves	its functional, as exact sums over the sample	what the engine already measures
read and thermal noise	independent every frame; its spread does not depend on the signal	the same curve's intercept: the spread left at zero signal	**measured** as the intercept's sum (rows 4 to 6); read, thermal and reset are not split at one temperature (C14); bits 0 to 3 at 499 to 500 flips per thousand are its footprint together with shot and quantization (**measured**)
quantization noise	uniform within one quantization step	exactly one step wide; the known part of the lowest bit	**measured** by the value histogram (row 22)
spatially correlated noise (rows, columns, banding, dust, hot pixels)	shared along a line or fixed at a pixel	per row and per column, the sum of the noise part; per pixel, a departure that holds across samples	rows and columns **measured** by the static line sums (rows 8 and 9, `noise_lines_set`); hot pixels and dust **measured**, none found (rows 20 and 21); each shared term's pattern is priced by the root noise of a box (E16, proved on the `noise_root` sim); the per-location cost map is theory (noise_sieve_tower.md §7)

Departure. Where the entropy history leaves the noise floor the functionals predict, something is there: a cell, a division, a drive (noise_sieve_tower.md §17). The coder spends bits on departure only, and the search looks only there. The same functionals that set the noise floor also set what the search counts as a departure. So the null draws, the climb and the sift all read against the integrated noise floor instead of against one sample-wide reading.

The coder that reaches F5. The counts come from the functionals, and the coder spends $\Sigma - \log_2 p$ over the stream against them. How close it gets depends on its arithmetic:

- **Exact arithmetic coding** (the interval stored as an exact integer of whatever width it needs) ends within 2 bits of that sum over the whole stream, however long.
- **Enumerative coding** in exact integers (the stream's index among every stream with the same counts) spends exactly $\lceil \log_2(n! \,/\, \Pi\, c!) \rceil$ bits: the F1 and F2 counts themselves, with no loss at all. It costs a big-integer multiply per symbol; the engine's exact integer is that machinery.
- **A fixed-width range coder** (32 or 64 bits) rounds each interval down to its width and loses a sliver per symbol, small but not zero. That is a rounding. It is the fast approximation to measure against the exact two instead of the design.

Two things the 2 bits does not cover.

- It is 2 bits from the *model's* length. If the functionals miss a real category, the loss is the mismatch between the model and the data, and F5 sits above the true lower bound by that much.
- Stating the model costs bits of its own: the functionals' parameters and the count tables. That is the two-part code (F3′), and a model is worth its bits only where it saves more.

It replaces the Rice coder's fixed block, k and escape (the audit's unmeasured constants) with the data's own counts.

Order of work.

1. Measure F2 on one sample to learn the Rice coder's own gap (the list at the end).
2. Take the photon transfer curve per sample from the frames: shot and read separated, as exact sums. Taken on 24 samples of the .kcr dataset: `noise_detector` takes the exact sums, and rows 1, 4 and 15 of the noise vector integration table hold the slope and the intercept. Read, thermal and reset stay one sum at one temperature (C14).
3. Write each category's functional, and the spatial ones per row, column and pixel. The rows, columns, planes, fixed pattern and stack term each have a pattern and a price in E16, proved on the sim. No functional is written.
4. Integrate them into a per-voxel count table: F5, measured.
5. Build the range coder on those counts and enter each dataset's compressed size against F5. Any gap left names a category not yet written.

2.6 THE TABLE

dataset	samples	raw (Fo)	compressed	of raw	hash tree's share	noise floor	gap	status	next
Cell tracking, 44b6	the 25 (first 25 44b6 training samples by name), 100 × 64 × 256 × 256 u16 each	20,971,520,000 bytes	8,809,343,524 bytes as `.iapx`, with a CRC-64 and no hash tree	42.0%	not in these (CRC-64, 8 bytes a sample)	F4 38.9% (6.229 bits a voxel), the mean of the 25 measured one by one (above); F1 to F3 not measured	3.1 points of raw: 0.492 bits a voxel, about 645 MB (C9)	measured, proved lossless (25 of 25 rebuilt, set CRC `091daa41e1aceb7e`)	Measure F2 per decomposition level on 44b6_0113de3b. The `.kcr` re-ingest is the next row.
Cell tracking, 44b6, as `.kcr`	the same 25	20,971,520,000 bytes	10,207,190,124 bytes as `.kcr`, sealed (set root `ad40e1d0…f53a39`, 757 s)	48.6%	in the compressed file, not taken apart	F4 38.9% (above)	not taken: the hash tree's share is in the compressed file	measured: 25 of 25 passed (the cell build plan, item 37); 44b6_587a1e22 fails its hash tree check (the noise vector integration table)	Take the hash tree's share apart, then F2 per level.
Cell tracking, 44b6_0113de3b alone	1	838,860,800 bytes	stream 340,189,016 bytes as the `.iapx` writes it	40.6% for the stream alone (6.49 bits a voxel)	none	F4 5.902 bits a voxel, 36.9% (photon transfer line 1.162 × level + 2.46, structure removed by neighbor differencing); bits 0 to 5 carry no linear generator (measured)	3.7 points of raw: 0.59 bits a voxel, about 310 KB a frame, 30.9 MB over the sample	measured	F2 on the stream, to learn the Rice coder's share of the 0.59. Then the stream variants in the next table.
RSNA knee, one series, unsigned	1 × 34 × 960 × 960	62,668,800 bytes	25,218,496 bytes, sealed `7ac2cb89…12eb2a`	40.2%	5.4% of the compressed file; 2.1% of raw at 960 columns	not measured	none	proved (rebuilt voxel for voxel, pixel for pixel, node for node)	F2 on it.

dataset	samples	raw (Fo)	compressed	of raw	hash tree's share	noise floor	gap	status	next
RSNA knee, one series, signed	1 × 24 × 640 × 640	19,660,800 bytes	11,319,416 bytes, sealed `3e70b8cb…8d9966`	57.6% (57.5% in the engine table, a rounding)	in the compressed file	not measured	none	proved	Why signed runs 17 points above unsigned: the offset by 2^15 into the unsigned 16-bit range, or the data itself. Compare F1 on both.
RSNA knee, test_series	15 series, 557 slices (the RSNA project)	599,191,552 bytes	192,020,272 bytes	32.0%	in the compressed files	not measured	none	measured in the RSNA project: 15 of 15 passed; the prove (696 ms) confirmed every hash tree node; set root `98b25a42…d7af979`	Their train set, 24,371 series: enter its total when its ingest completes.
DICOM headers (the side information)	the knee series	none	deflated by our own LZ77 and Huffman coder	within about 1% of zlib level 9	none	zlib 9 is a reference, not a lower bound	about 1%	measured, round-tripped through our inflate and zlib	none

The hash tree is overhead above any lower bound: it is the price of every node being locatable, and a lossless coder does not undo it. Measured on the knee at 5.4% of the compressed file, it scales as 16/X of raw from the rows (X the row length), and it runs larger on narrow rows.

2.7 WHAT HAS BEEN TRIED ON THE COEFFICIENTS

All on 44b6_0113de3b, against the 340,189,016-byte stream, each variant decoded back and checked (ledger).

what	stream change	status
radix around the line of 2, 4, 8 neighbors along x	+13.9, +18.1, +19.7 MB	refuted: the lifting transform already took the spatial correlation
the same along y	+6.4, +11.3, +12.1 MB	refuted
the same along t	+1.35, +2.10, +0.18 MB	refuted: real temporal structure, but the pivot shear costs more than it takes
each level's subbands row by row	− 1,157,533 bytes	measured: grouping by level helps; not built into the coder
shells of squared radius, each swept by angle	+658,073 bytes	refuted for the angle sweep only; the golden order (k(φ − 1) mod 1, `cell_tracking/maint/emit_spiral_table.py`) is untested
2 along t, each level apart	− 1,987,962 bytes	measured on this sample; hurts on max others (the e − 1 rows below)
2, 4, 8 along t, rotated onto each level's line	− 1.88, − 1.63, − 1.47 MB	refuted: worse than no rotation at every n
pairs along t over rectangles, the savings ratio	1.7174 here, then − 8.29, 0.44, − 73.2, − 1.60 on four more samples	refuted: a coincidence

The Rice coder's own numbers are unmeasured constants in the engine table's audit: `COMPRESSION_BLOCK` 64, 64 blocks a chunk, `COMPRESSION_K_BITS` 5, `COMPRESSION_ESCAPE` 24. They are chosen, not measured, and measuring each against F2 is how the coder's share of the gap is found.

2.8 BUILT IN THE ENGINE, NOT YET IN THE COMPRESSION PATH

Each part is in `orior/src/engine`, with its proof in the engine ledger or engine_table.md. A compressed file's size owes nothing to any of them.

part	what it does toward the lower bound	proof	status
The lookup-table step (M10, A13; C21)	a pointwise function evaluated over the lane's alphabet once and read back as a lookup table; one table read through another is their composition. In a two-part code it is the model's part (F3′), stated once and read per lane	`record_table_test` (`test/engine/record_table_test.cu`): x^2 and $\lvert x - 30000\rvert + 100$ over the 16-bit alphabet, 4,096 lanes, the table and the ops agreeing word for word on device and host; a table read through a table against the composed ops; 15 checks, 0 failed	proved for the lookup-table step; square root and division as tables untested; not in the compression path
The lifting transform's edges (M20, A14)	a bijection on a level's low index bits inside the reversible stream; any other function is made reversible by the embedding (x, y) → (x, y ⊕ f(x))	`tower_edge_test` (`test/engine/tower_edge_test.cu`), 25 checks, 0 failed, and the `root_universal` sim, 1,268 checks, 0 failed: lift, code, decode and lower exact on 48 of 48	proved; `engine.cu` passes no edges. A keyed random edge costs 6.810 to 12.608 coded bits a voxel on level 0 at widths 2 to 12, against 6.802 with none; a fitted edge not measured; its place in the `.kcr` is Doug's call
The root noise of a box (E16)	for each shared term (rows 8, 9, 10, 18 and 25 of the noise vector integration table), a pattern of floor means and its residual, which return the box exactly; the root is the term whose pattern and residual code smallest against the box	the `noise_root` sim, 72 checks, 0 failed: at variance 1,024 every term is the root, saving 32 to 89 per mille of the box's bits	proved on the sim; no driver part over real boxes, and the pattern is not stored in the compressed file
The lattice price (`engine_lattice_bits`, engine.h)	the coder's bits for a lattice of ints, lifted by `tower_lift` and coded by `compression_encode`	the engine build compiles it	built and not yet run: nothing calls it until a driver part does

2.9 HOW TO CLOSE THE GAP, IN ORDER

1. **Measure F2 on one sample** (44b6_0113de3b). The gap between the compressed file and F2 is the Rice coder's own share: its block size, its k and its escape against the counts. That share of the gap is recoverable without changing the lifting transform.

2. **Measure F3 with the contexts the ledger already points at.** Grouping by level (−1.16 MB) and pairs along t (−1.99 MB on this sample) are contexts. F3 says what the best such context can take, and so whether to build one.

3. **Settle F4.** Measured on 44b6_0113de3b at 5.902 bits a voxel (36.9%): bits 0 to 5 carry no linear generator, and the photon transfer curve with structure removed gives gain 1.162, read noise about 1.6, and no dark offset. Measured on the 25 at 6.229 bits a voxel (38.9%), gain 0.986 on average. Still to do: the photon transfer line above signal level 300, where it is assumed and not read; the constant regions and the spatially correlated category (44b6_267148e4, 668e0cc7, 5740d24b) as functionals; the bent fits (12dfb391, 53f95252); the z correlation near 0.12 at the dimmest levels; nonlinear generator classes on the low planes. Read on 24 samples of the .kcr dataset, each measured and none yet a functional: the constant region (row 24 of the noise vector integration table), the dim z correlation (row 25) and the bending transfer curve (row 26).

4. **The noise functionals** (the plan above): the photon transfer curve, each category's functional, F5 integrated, and the range coder that reaches it.

5. **Build the coder the measurements pick**, and enter every dataset's new size in this table with its noise floor beside it.

3

THE BUILD PLAN: COMPRESSION

Purpose: The compression item of the build plan. The order of work on the lower bound is in compression_table.md ("The plan" and "How to close the gap").

Scope: the one compression finding the build plan records under the engine's item 36 (the engine workbook's build_plan.md).

3.1 FOUND, 24 SEPTEMBER

- Two builds of 44b6_0113de3b.iapx measure 341,008,288 and 396,922,152 bytes, a difference of 16.4%. The hash tree's cost is 5.4%, and the rest is not explained. Not investigated.

4

LEDGER

Purpose: Every measurement, in the order it was taken, with the samples it ran on, the number, and what it settled. No result is taken twice and none is quoted without its run.

Scope: runs of the .kcr codec, its variants, and the measurements of the lower bound it answers to. Samples are named by their id; "the 25" means the first 25 44b6 training samples by name.

4.1 2026-09-21

4.2 THE .KCR CODEC

what	samples	result	settles
lifting-transform codec, whole sample as one object	the 25	42.0% of raw; every file rebuilt voxel for voxel on the device and pixel for pixel off it	lossless compression of the frames (proved)

4.3 THE RESIDUAL'S CODE LENGTH: RADIX AROUND A LINE, ROTATION, SPIRAL

Stream sizes against the residue as the .kcr writes it, sample 44b6_0113de3b, 340,189,016 bytes. Every alternative was decoded back and checked.

what	result	settles
radix around the line of 2, 4, 8 neighbors along x	+13.9, +18.1, +19.7 MB	refuted: the lifting transform already took the spatial correlation
along y	+6.4, +11.3, +12.1 MB	refuted
along t	+1.35, +2.10, +0.18 MB	refuted: real temporal structure, but the pivot shear costs more than it takes
each level's subbands row by row	− 1,157,533 bytes	grouping by level helps

what	result	settles
each level's subbands in shells of squared radius, each shell swept by angle	+658,073 bytes	this is NOT the golden spiral: it sweeps each shell in angle order, neighbor after neighbor, where the golden order (cell_tracking/maint/emit_spiral_table.py) ranks a shell by k(φ − 1) mod 1 so no direction repeats at any prefix. It refutes the angle sweep only; the golden order is untested here
2 along t, each level apart	−1,987,962 bytes	helps on this sample
2, 4, 8 along t rotated onto each level's line	−1.88, −1.63, −1.47 MB	refuted: rotation is worse than no rotation at every n

4.4 THE E − 1 RATIO

The ratio of the two savings (pairs along t over rectangles) is 1.717412 on 44b6_0113de3b, near e − 1 = 1.718281.

sample	rectangles saved	pairs along t saved	ratio
44b6_0b24845f	478,803	−3,970,121	−8.29
44b6_0c582fdc	552,737	−241,482	0.44
44b6_0db75fae	268,296	−19,646,805	−73.2
44b6_12dfb391	469,282	−751,835	−1.60

Settles: refuted. The first ratio is a coincidence, and pairing along t hurts on max samples.

4.5 2026-09-23

4.6 A SYNTHETIC TEST OF THE LOWER BOUND

A synthetic set with a known generator, against which every code length can be checked: 32 frames of 32 × 32 (32,768 voxels); each voxel's noise is Binomial(2s + 16, ½), drawn as the popcount of xorshift64 bits, where s is a disc of signal peaking at 40; a fixed-pattern offset in [0, 8) per pixel. Raw is 7 bits a voxel. Every length is the bit length of an exact integer (a product of binomial coefficients, or a multinomial), with no float. The script is scratch (`experiment.py`), not in the tree.

what	samples	result	settles
raw	the synthetic set	229,376 bits	the reference
1. the true bound: the generating model, told its parameters	the synthetic set	120,441 bits	the bound for this source
2. ours: per pixel, the offset and the spread fitted by exact code length in one pass, 9 bits of parameters a pixel counted in	the synthetic set	128,805 bits, 7% above the true bound, all of it the parameters	the functionals reach the bound, less what stating them costs
3. theirs: the previous frame as the prediction, one global table of residues coded enumeratively	the synthetic set	144,386 bits	

what	samples	result	settles
4. theirs taken through our functionals: the frame differences coded per pixel by exact length	the synthetic set	145,093 bits	a model that differences frames doubles the noise it must code; our functionals do not rescue it
5. the shortest of 2, 3 and 4	the synthetic set	128,805 bits: row 2	any model is a candidate, and the pipeline retains the shortest; the bound it reaches is ours
6. the noise reverse engineered: the xorshift seed solved from the parity of each voxel's noise, by one GF(2) elimination	the synthetic set	the seed recovered exactly; 0 of 32,768 parity bits wrong when rebuilt	noise from a linear generator is 64 bits and a program, not a bound
7. the same solver on os.urandom noise	the synthetic set	16,346 of 32,768 wrong: half, a coin	true noise has no seed to find; what remains is the bound

4.7 THE NOISE FLOOR ON 44B6_0113DE3B

Read from the frames through `cell_tracking/maint/zarr_frames.py`, about 78 s a read. The scripts are scratch (`linear_complexity.py`, `conditional_floor.py`, `photon_transfer.py`, `coherence.py`), not in the tree. The tables in full, and the algebra behind each reading, are in compression_table.md.

what	samples	result	settles
Berlekamp–Massey controls	os.urandom, 512 × 256 bits and 2,048 × 100; xorshift64, 64 seeds × 256 bits	urandom mean L 128.25 and 50.19, 0 low; xorshift L = 64 on all 64	the test separates a linear generator from random bits every time
linear complexity, bits 0 to 5	44b6_0113de3b: 4,109 voxels in time (n 100), 1,024 rows of frame 50 (n 256), 4 raster runs (n 8,192)	mean L 50.22 to 50.24, 128.14 to 128.26, 4,096 to 4,097; 0 low in every direction	the low six bit planes carry no linear generator; indistinguishable from os.urandom (nonlinear generators untested)
linear complexity, bits 6 to 11	the same	in time, mean L falls 48.72 → 43.48 → 36.68 → 26.56 → 19.43 → 1.79, low 201 → 4,024; along rows from bit 7	structure begins at bit 6; bit 11 is almost constant
noise variance by signal level, from successive frame differences	44b6_0113de3b, 4,109 voxels × 100 frames, by mean decile	from σ 7.6 at level 47.2 (variance / mean 1.22) to σ 218.7 at level 523.0 (variance / mean 91.43)	variance grows far faster than shot noise above the dimmest decile: the frame-to-frame change there is mostly cells moving, which the tracks predict
the conditional noise floor, motion left in	the same	7.403 bits a voxel (46.3%) by each voxel's σ; 7.687 (48.0%) by empirical counts per σ bin	an upper bound for a temporal-only model; above the stream's 6.49

what	samples	result	settles
one frame alone against its 4 neighbors, zeroth-order	frame 50, 462,400 voxels	6.287 bits a voxel (39.3%)	a reference only: it uses neighbors on both sides, and no coder can spend it
the shot-limited noise floor, first estimate	extrapolated from the dimmest decile: variance = 1.22 × level at every decile	about 6.00 bits a voxel, 37.5% of raw	superseded by the two rows below
the photon transfer curve, motion removed in time	44b6_0113de3b, 41,528 voxels (every 101st) × 99 differences, each binned by its pair's level; σ from the median absolute deviation	variance / level 1.15 to 1.35 from level 24 to 104, first and second differences agreeing; 1.55 at 120, 3.17 at 312, 45.85 at 504, a plateau near 170,000 past 700	shot noise in the dim range; above 120 the change is in max frames, and the median cannot take it; the all-bins fit (138.4 × level − 23,437, 4.97 bits) is not a curve and is discarded
the same, structure removed by neighbor differencing	44b6_0113de3b, 10 frame pairs of whole frames, 41,943,040 voxel-frames; the frame difference against its x, y and z neighbors	corr(d, d_x) 0.007 at level 24, 0.348 at 104, 0.680 at 200, 0.985 at 1000; var(d − d_x) / 4 at 1.17 to 1.23 × level from 40 to 200; fitted variance = 1.162 × level + 2.46	the bright excess is structure instead of noise; the noise is shot at gain 1.162 with read noise about 1.6 and no dark offset; the z correlation of 0.12 at the dimmest levels is not explained
F_4, measured	the same	5.902 bits a voxel (36.9%) by the photon transfer line; 5.979 (37.4%) less the x neighbor, robust; 6.008 (37.6%) best axis, plain; 6.913 (43.2%) structure in	one frame given everything else costs 5.90 bits a voxel; the stream's 6.49 is 0.59 above it: 3.7 points of raw, about 310 KB a frame

4.8 THE NOISE FLOOR ON THE 25 SAMPLES

The same two passes on the other 24; the per-sample table is in compression_table.md, and each reading cites its identity there.

what	samples	result	settles
the photon transfer curve, motion removed in time	the other 24, 4 at a time, 714 s, 0 failed	the per-bin robust noise floor, structure partly in: 5.289 (44b6_341df25f) to 8.422 bits a voxel (18ced818)	the time pass alone cannot part structure from noise on bright samples; superseded by the coherence pass
the same, structure removed by neighbor differencing	the other 24, 2 at a time, 874 s, 0 failed	gain 0.821 to 1.162, mean 0.986; intercept −11.74 to 21.08, near zero on max; shot-only F_4 5.085 to 6.973 bits a voxel, mean 6.229 (38.9%); the neighbor bound mean 6.243 (39.0%); structure in, mean 7.215 (45.1%)	one sensor noise model across the dataset, about one count a photon; the dataset's noise floor is 38.9% of raw, 3.1 points (0.492 bits a voxel, about 645 MB) under the compressed files' 42.0%

what	samples	result	settles
the constant regions against the photon transfer line	44b6_0db75fae, 267148e4, 5740d24b	the neighbor bound 32.9%, 32.6%, 36.9% under the shot-only 34.5%, 35.6%, 38.4%	voxels that never change cost nothing, and the photon transfer line does not see them; the three are the engine ledger's constant-region samples
dim noise correlated in space	the 25, corr x of the frame difference at level 40	below 0.11 on 21 samples; 0.761 on 44b6_267148e4, 0.354 on 668e0cc7, 0.150 on 5740d24b	a spatially correlated noise category on those three, not identified
bent fits	44b6_12dfb391, 53f95252	intercepts −11.74 and 21.08	a straight line is not quite their law between level 40 and 200

BIBLIOGRAPHY

[1] A. N. Kolmogorov. *A new metric invariant of transitive dynamical systems and automorphisms of Lebesgue spaces.* Dokl. Akad. Nauk SSSR 119(5):861-864, Math-Net.Ru dan22922. 1958.

[2] C. E. Shannon. *A Mathematical Theory of Communication.* Bell System Technical Journal 27, 379-423 and 623-656, July and October 1948.